

STANSE: Bug-finding Framework for C Programs

Jan Obdržálek, Jiří Slabý and Marek Trtík

Masaryk University, Brno, Czech Republic
{obdrzalek,slaby,trtik}@fi.muni.cz

Abstract. STANSE is a free (available under the GPLv2 license) modular framework for finding bugs in C programs using static analysis. Its two main design goals are applicability on large software projects like the Linux kernel and extendability with new bug-finding techniques with a minimal effort. Currently there are four bug-finding algorithms implemented within STANSE: AUTOMATONCHECKER checks properties described in an automata-based formalism, THREADCHECKER detects deadlocks among multiple threads, LOCKCHECKER finds locking errors based on statistics, and REACHABILITYCHECKER looks for unreachable code. STANSE has been tested on the Linux kernel, where it has found dozens of previously undiscovered bugs.

1 Introduction

During the last decade, bug-finding techniques based on static analysis have finally come of age. One of the papers to really stir interest was [2], showing that static analysis can efficiently find many interesting bugs in real-world code. This work eventually led to a successful commercial tool called COVERITY [8]. Over the years, several other successful tools, like CODESONAR [7] or KLOCWORK [10] appeared. However, such fully-featured tools are neither free to obtain, nor is their code available (e.g. for developing new algorithms or tailoring the existing tools to specific tasks). The existing free tools are usually severely limited in what they can do (e.g. UNO [13], SPARSE [12], SMATCH [11]). One notable exception is FINDBUGS [5, 9], a successful tool working on Java code. STANSE is intended to fill this gap for the C language. It can be seen in two ways:

1. STANSE is a robust framework (written predominantly in Java) for implementing diverse static analysis algorithms. An implemented algorithm can be immediately evaluated on large real-world software projects written in C as the framework is capable to process such projects (for example, it can process the whole Linux kernel). An implementation of such an algorithm within the framework is called a *checker*.
2. As STANSE already contains four checkers, it can be also seen as a working static analysis tool.

The tools mentioned so far, including STANSE, provide may analysis. There is also wide range of static analysis tools providing must analysis. Successful tools

Nechapu proc recenzi
 zenti chteji srovnani
 STANSE se SOOTem?
 Podle me se jedna o
 zcela nesrovnatelne veci.
 Nenapada Vas, co maji
 spolecne, co by se melo
 okomentovat/porovnat?

like PEX, KLEE, EXE, or SAGE are of that kind. Since may and must analyses are complementary, they are very different to STANSE in basic principles.

The rest of the paper is structured as follows. Section 2 describes the functionality provided by the framework, while Section 3 is devoted to the four existing checkers. Section 4 presents some results of STANSE on the Linux kernel. The last section summarizes the basic strengths of STANSE and mentions some directions of its future development.

2 Framework Functionality

The STANSE framework is modular and fully open. It is designed to allow static analysis of large software projects like Linux kernel. Further it is aimed to reduce effort when implementing a new static analysis technique. Architecture of the framework is depicted in Fig. 1. As can be seen, the tool consists of standard parts for these kinds of tools. Apart from that there are also parts which are not common to all static analyzers or are specialized in some way. In the following subsections we describe those.

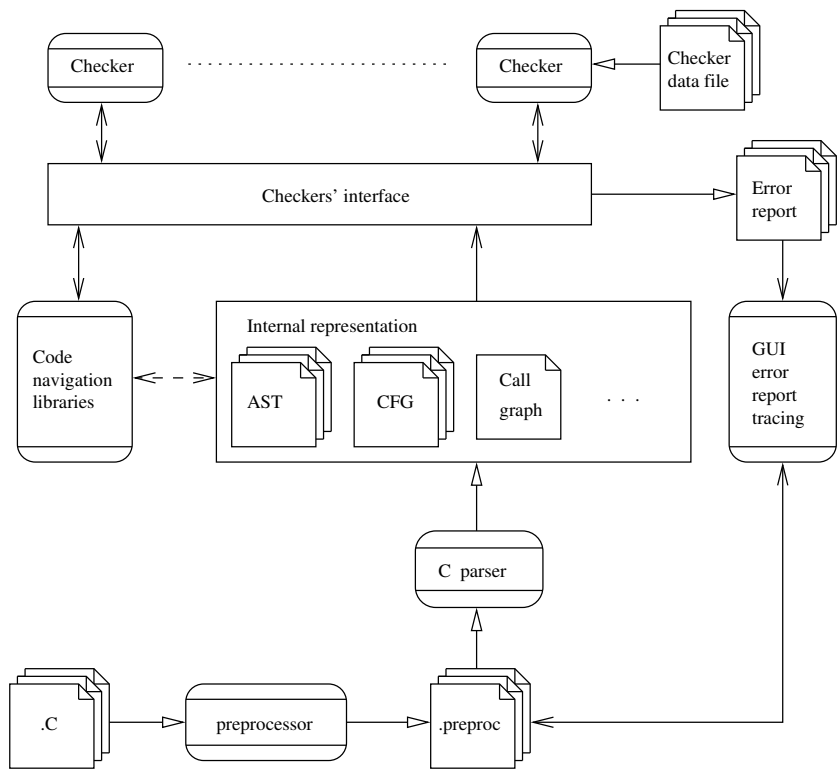


Fig. 1. STANSE framework architecture.

2.1 Configuration

The STANSE framework contains data structures capturing a configuration of an analysis to be performed. We always need to know what source files to analyze and by what checkers. That information we call *configuration*.

There are several ways how to tell STANSE what source files to analyse. Besides the standard possibilities (a single given file, all files from a given directory, all files listed in a given file), STANSE can also derive all the necessary information from a project Makefile. In this case, STANSE also remembers compiler flags for preprocessing purposes. This functionality is inspired by the SPARSE [12] tool. At the bottom left corner of Fig. 1, we show a list of .C files which are identified in the configuration.

We also need to specify checkers which should be applied to configured source files. We can see the list of selected checkers on the top of Fig. 1. There can be many checkers running simultaneously in STANSE. But they do not see each other. They proceed independently. Checkers themselves might require their own configuration files. They are depicted at the top right corner of Fig. 1.

The configuration can be passed to the STANSE framework via command line arguments. User can also add them one by one in the graphical interface.

2.2 Parsing Source Files

The STANSE framework can process source files written in C. More specifically the ISO/ANSI C99 standard and most of the GNU C extensions. This allows STANSE to process software projects like the Linux kernel. We are currently working on support of other languages, in particular C++. We have a prototype implementation, which is included in the distribution already. The parsing pipeline, including a preprocessing the source files by the standard GNU C preprocessor, is depicted at the bottom in Fig. 1. It is important to note that we do not parse all the sources in a sequence one by one as they appear in a configuration. Since we use streaming (discussed later), the pipeline is applied several times independently for individual source files.

2.3 Program Internal Representation

When the code is parsed, we have *internal structures*: a call graph among functions, a control flow graph (CFG) for each function, and an abstract syntax tree (AST) for the whole file. Subtrees in the AST are referenced from appropriate CFG nodes. We show these structures in the middle of Fig. 1. All these structures can be dumped in a textual or graphical form.

Since we aim at large software projects consisting of hundreds or thousands of modules (compilation units), it is often impossible in practice to store the corresponding internal structures in the memory all at once. The STANSE framework therefore applies automatic *streaming* of the internal structures. This is currently performed on a module basis. Instead of parsing all source modules at the beginning, a module is streamed in only when STANSE needs to access some

Tady je potreba doplnit, proc jsme napisali vlastni parser, proc jsme nepouzili GCC, a konecne, proc jsme nepouzili LLVM (CLANG). Take by bylo dobre dodat odkazy na GCC a LLVM do bibliografie.

internal structure corresponding to the module. In other words, the internal structures are constructed in a lazy manner.

If the memory occupied by internal structures exceeds a given limit, some internal structures have to be freed before another module is streamed in. The structures to be freed are selected by the LRU (least recently used) approach: STANSE discards all internal structures of the module whose structures are not accessed for the longest time. If the discarded structure is accessed again later, the corresponding module is streamed back in. Both laziness of internal structures and streaming are completely invisible to checkers. A purpose of the streaming is to allow checkers to assume that all sources are presented in the memory at once.

In current implementation of streaming each source file streamed out from the memory is completely discarded. STANSE does not backup already parsed internal structures into auxiliary files before discarding. As a consequence, when internal structures of the discarded file are needed again, STANSE starts the parsing pipeline of the file from scratch to recreate requested internal structures. Although loading of previously parsed internal structures from auxiliary files would speed up the streaming process, a profiling of STANSE's performance on the Linux kernel have not shown streaming to be a performance bottleneck. An usage of such auxiliary files can be implemented, if streaming performance became a performance problem in a future.

2.4 Pointer Analysis in Stanse

When analysing programming languages involving pointers, it is almost always necessary to use an appropriate kind of pointer analysis. Since C is such a type of programming language, the STANSE framework should provide a pointer analysis. But there are many different kind of analyses. Many of them are flow-insensitive, some are context-insensitive, some are field-insensitive, and so on. There are many pointer analyses that combine mentioned properties to balance between precision and performance.

It is difficult to predict, what pointer analyses are the most likely to be used in checkers implemented in STANSE in future. Therefore the STANSE framework does not contain any particular implementation of pointer analysis. Instead, we have implemented *Steensgard's* flow-insensitive and context-insensitive points-to analysis and also *Shapiro-Horowitz's* points-to analysis, which allows parametrization between Steensgard's and Andersen's points-to analyses. *Shapiro-Horowitz's* points-to analysis is flow-insensitive but can be context-sensitive. Both analyses are currently implemented as regular checkers. These analyses can be used in other checkers by loading particular analysis as a sub-checker.

The decision of implementing pointer analyses in separate checkers has one important negative drawback. When two or more checkers are run simultaneously in STANSE and they use the same kind of pointer analysis (as their own sub-checker), then the pointer analysis is executed for each checker independently - i.e. several times. If the points to analyses were implemented inside the STANSE framework, this won't happen.

The reason for implementing points to analyses in checkers instead of in the STANSE framework is just because we have not made a final decision, which pointer analyses are the best to be put into the framework. The current solution with checkers is a temporary one.

2.5 Matching Language Constructs

Checkers, as implementations of many static analyses, usually work with some syntactical subset of language constructs. On these constructs the checkers define how sets of their abstract states change. On the remaining language constructs the abstract states do not change at all, i.e. state transformation function on such the states is just identity.

It depends on particular checker which constructs affect sets of its abstract states and which do not. Therefore STANSE offers checkers a parametric specification of language subset, which is important for them. It works as follows.

Since AST seems to be very useful internal representation of an analysed code for many types of many analyses, the STANSE framework defines a simple language speaking about structure of AST. In this language a checker can specify, what kinds of sub-trees of AST the checker is interested in. A basic construct in this simple language is so called *pattern*. It is an XML description of particular subtree in AST, which may contain special nodes - keywords in this language, which allow parametrization of shape and also appearance of particular AST nodes in the tree. Therefore, a pattern is a specification of whole set of AST sub-trees. And collection of patterns constitute the parametric specification of language constructs a checker is interested in.

The STANSE framework contains a library, which provides loading of patterns and also functions scanning AST of analysed functions to find matching instances of patterns in the ASTs of the analysed functions.

When a checker knows where in an AST matches which pattern, it then simply knows where should be applied which action. This greatly simplifies implementation of checkers.

A similar system of recognition of specific language constructs was introduced in [1]. Their language METAL allows parametrization of C grammar, instead of parametrizing AST subtrees as STANSE's patterns do. Nevertheless, the purpose of both approaches is the same.

doplňit citaci METALu

2.6 Traversing Internal Representation

Although a checker may need to work with the internal structures in an arbitrary way, most of checkers walk through CFGs using some standard strategy. To prevent unnecessary reimplementations, the most important and heavily used traversal methods are implemented directly inside the framework. With this functionality, one can implement a new checker (or its part) by specifying

- whether it should go through CFGs forwards or backwards, breadth-first or depth-first,

- if the interprocedural walk-through should be performed or not (if not, the function calls are ignored), and
- a method (callback) to be called for each visited node in a CFG.

This makes implementation of new algorithms extremely simple.

For example, when a checker must implement a forward flow-sensitive analysis, then it asks the STANSE framework to traverse paths in CFGs in forward depth-first manner. This can be implemented by a single call to a library function `traverseCFGToDepthForward`, which takes particular CFG as an argument and also subclass of STANSE library class `CFGPathVisitor`, where the checker implements action, which should be taken whenever CFG node is visited (already in requested order). The checker implements the action in a method `visit` of the subclass.

Another example is support for interprocedural analysis. It is common for interprocedural analyses to build function summaries. Unfortunately, these summaries might differ from analysis to another. It is difficult in general to build a single universal representation of function summaries, which will suit to any kind of analysis. Therefore, instead of trying to provide one, the STANSE framework provides libraries, which are likely to be helpful in constructing many different kinds of summaries for different analyses. Those libraries provides automated mapping between arguments and formal parameters in call sites (library `ArgumentPassingManager`) and automated mapping of return values back to calling functions (library `ReturnValuePassingManager`). In addition, for those interprocedural analyses, which do not construct summaries at all (and rely on function inlining), STANSE provide an automated traversal between different CFGs according to function calls. This can be again done by a single call to STANSE library from a checker (e.g. `traverseCFGPathsForwardInterprocedural`).

The discussed functionality is shown to the left of internal representation in Fig. 1.

2.7 Concept of Checkers

In the STANSE framework a checker is an implementation of some concrete static analysis technique. Each checker has `an` access to a shared internal structure of analysed source files. They are also provided with an access to the algorithms providing navigation in those structures. This is done by an interface between checkers and internal structure and libraries of the framework. The interface is depicted in Fig. 1 right below the checkers.

The checkers are integrated in the framework of STANSE using concrete factory design pattern. Therefore, to insert a new checker to the framework one needs to implement generic checker interface and register it to the checkers' factory of the framework. Then it gains a full access to the features of the framework accessible through the discussed interface.

The STANSE framework allows checkers to show a progress of their activity in an uniform way. Each checker is passed an instance of a `ProgressManager` class into which the checker just sends its progress messages. The checker does

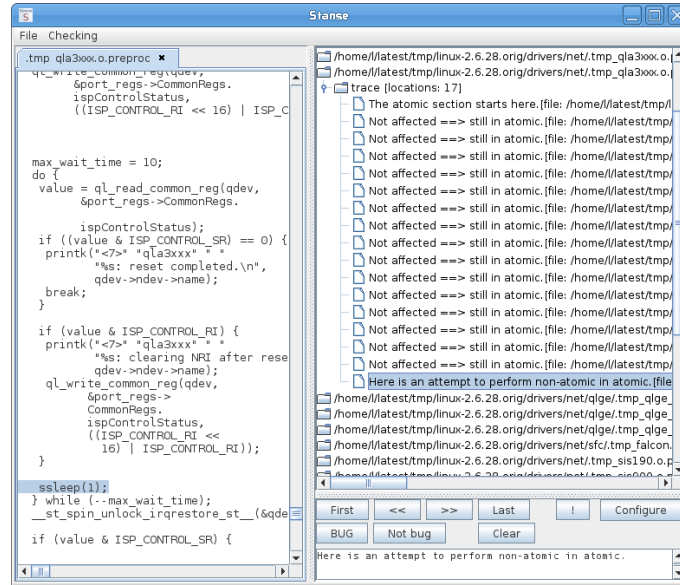


Fig. 2. Error trace browser in the STANSE GUI.

not need to care where the messages go (whether to a particular GUI window or command line) and how to show them there. It simplifies the implementation of checkers while receiving a feel of an uniform implementation of all the checkers.

2.8 Processing Errors

Once a checker finds an error, it reports it back to the framework as a commented *error trace* - a path in the analysed code demonstrating this error. The STANSE framework provides a data type for describing an error. In the framework there are then several possibilities how to present the error traces back to the user of STANSE: they can be printed to the console, displayed using a built-in error trace browser in the GUI (see Fig. 2), or saved to an external file in XML format. This XML file has a wide variety of possible applications. For example, we supply a tool transforming the XML file into an SQLITE database. The database is complemented with a web interface allowing to browse errors in the database via a web browser. Using the web browser or the built-in graphical error browser, one can mark errors as real bugs or false positives. STANSE also provides various statistics of errors like number of errors per checker, frequency of errors of the same kind, percentage of false positives (based on user feedback).

Error reporting and tracing pipeline is depicted to the right of interface and internal representation of Fig. 1.

3 Checkers

In this section we briefly describe the four currently available checkers.

AutomatonChecker is heavily influenced by [2]. It takes, as input, a set of finite-state automata that describe some properties which we want to check. Properties like locking discipline, interrupt management, null pointer dereference, dangling pointers and many others can be handled this way.

Compared to the implementation described in [2] and [4], our technique differs in several aspects. In particular, we do not use metacompilation, automata are not input-language specific (a pattern matching is used instead), and the interprocedural analysis is done in the context of a single input file.

LockChecker accounts statistics about variable accesses. It also tracks which locks are locked while each of the variable is accessed. Then, with combination of this information, it counts in how many cases each variable is accessed while some lock is held. If, for instance, some variable is changed 99 times while some lock is held and in one case it is not, this is reported as possible error. The boundary is currently set to 70, so that at least 70% of accesses must be under locks. The rest (30%) is then reported. This work is based upon [3].

ThreadChecker aims to check for possible deadlocks in concurrent programs. The technique is based on the notions of locksets of [6] and deadlock detection by looking for cycles in *resource allocation graphs (RAGs)*. THREADCHECKER first tries to identify the parts of the code which can run in parallel, as different threads. Then it builds a set of *dependency graphs* for each such thread. A dependency graph statically represents the possible locksets during one execution of a thread. Dependency graphs are then combined and transformed to RAGs. If some RAG contains a cycle, an error is reported.

ReachabilityChecker searches CFGs for unreachable nodes. These are then reported as warnings or errors, depending on importance (e.g. superfluous semicolons are less important than unused branch). The primary goal of REACHABILITYCHECKER is to demonstrate the simplicity of a new checker implementation. With a help of the framework features described in Subsection 2.6. The code of the checker has less than 200 lines including the mentioned error/warning classification and many strings and comments.

4 Results on Linux Kernel

We have several reasons to choose the Linux kernel for testing STANSE: the kernel is a large and freely available codebase, it fully exercises most features of ISO/ANSI C99 and GNU C extensions, it is under constant development (there is a constant income of new bugs), and absence of bugs is of great concern.

Checker	Automaton	Errors			Real/classified error ratio
		Found	Real	False pos.	
AUTOMATONCHECKER	Pairing	266	65	143	31.3 %
	Pointers	86	48	37	56.5 %
	Deadlocks	35	16	18	47.1 %
LOCKCHECKER		13	6	7	46.2 %
THREADCHECKER		20	9	11	45.0 %
REACHABILITYCHECKER		31	31	0	100.0 %
Overall		451	175	216	47.9 %

Table 1. STANSE results on the Linux kernel version 2.6.28.

We applied STANSE with the four checkers described in the previous section. As a base, we used kernel version 2.6.28. AUTOMATONCHECKER uses three automata describing the following kinds of errors:

- incorrect pairing of functions (imbalanced locking, reference counting errors)
- bugs in pointer manipulation (null dereference, dangling pointers, etc.)
- deadlocks caused by sleeping inside spinlocks or interrupt handlers

The running time of STANSE on a common desktop machine with two 2.5 GHz cores and 4 GiB of memory is under two hours. The memory usage of the Java process oscillates between 400 and 1000 MiB. The number of errors found by the checkers can be found in Table 1. Let us note that REACHABILITYCHECKER actually found 751 errors, but 720 of them are of low importance (including 696 superfluous semicolons) and they are omitted from our statistics.

We have manually analysed all the found errors and classified them as real errors or false positives (with an exception of 60 errors found by AUTOMATONCHECKER where we are not able to decide in a short time whether it is a false positive or not). **It is important to note, that the checkers do not produce any false negatives (assuming there is no bug in the checkers' implementation). They all implement may analyses and as a such they can not miss a defect underlying their specifications.** The numbers of real errors, false positives and the ratio of real errors to all classified errors can be also found in Table 1. The overall ratio of real errors to all classified errors is not high: 47.9%. However, STANSE in the current version does not have any thorough false positive filtering technique.

More than 70 of the 169 real errors have been reported to kernel developers and fixed in the following kernel releases (the rest have been independently discovered and reported by someone else or the incorrect code disappeared from the kernel before we finished our evaluation of found errors). Some of the reported bugs remained undiscovered for more than seven years (for illustration, see our report at <http://lkml.org/lkml/2009/3/11/380>).

We have reported another 60 bugs found by STANSE in the subsequent versions of the kernel. This number is increasing every month.

5 Conclusions and Future Work

STANSE is a free Java-based framework for simple implementation of bug-finding algorithms based on static analysis. The framework can process large-scale software projects written in ISO/ANSI C99. STANSE does not currently use any new techniques – its novelty comes from the fact that (to our best knowledge) there is no other open-source framework with comparable applicability and efficiency. We note that more than 130 bugs found by STANSE have been reported to and confirmed by Linux kernel developers already.

Future Work We plan to improve the framework in several directions. We are currently working on C++ support. Further, we plan to provide STANSE in the form of a plug-in for Eclipse and NetBeans IDEs. A lot of work can be done in the area of automatic false alarm filtering and error importance classification. Independently of new features development, we would like to speed up the framework as well. More precisely, we intend to replace the current parser written in Java by an optimized parser written in C, to replace the XML format of internal structures by a more succinct representation, to add a support for function summaries, etc.

Acknowledgements Jan Kučera is the author of the THREADCHECKER. We would like to thank Linux kernel developers, and Cyrill Gorcunov for STANSE alpha testing and useful suggestions. All authors are supported by the research centre Institute for Theoretical Computer Science (ITI), project No. 1M0545.

References

1. A. Chou, B. Chelf, D. Engler, and M. Heinrich. Using meta-level compilation to check FLASH protocol code. *ACM SIGOPS Operating Systems Review*, 34(5):59–70, 2000.
2. D. Engler, B. Chelf, A. Chou, and S. Hallem. Checking system rules using system-specific, programmer-written compiler extensions. In *OSDI'00*, pages 1–16, 2000.
3. D. Engler, D.Y. Chen, S. Hallem, A. Chou, and B. Chelf. Bugs as deviant behavior: A general approach to inferring errors in systems code. volume 35. ACM, 2001.
4. S. Hallem, B. Chelf, Y. Xie, and D. Engler. A system and language for building system-specific, static analyses. In *PLDI'02*, pages 69–82. ACM, 2002.
5. D. Hovemeyer and W. Pugh. Finding bugs is easy. In *OOPSLA'04*, pages 132–136. ACM, 2004.
6. J.W. Voun, R. Jhala, and S. Lerner. RELAY: static race detection on millions of lines of code. In *ESEC-FSE'07*, pages 205–214. ACM, 2007.
7. CODESONAR. <http://www.grammatech.com/products/codesonar/>.
8. COVERITY. <http://www.coverity.com/products/>.
9. FINDBUGS. <http://findbugs.sourceforge.net/>.
10. KLOCWORK. <http://www.klocwork.com/products/>.
11. SMATCH. <http://smatch.sourceforge.net/>.
12. SPARSE. <http://www.kernel.org/pub/software/devel/sparse/>.
13. UNO. <http://spinroot.com/uno/>.

A Availability

Website: <http://stanse.fi.muni.cz/>
Version: 1.2.0
Download: <http://stanse.fi.muni.cz/download>

The source code is also available. STANSE is distributed under the GPL v2 license. Currently, the tool is used mainly by the developers and students at the Masaryk University. It is also used by programmers in a partner software company. Several other kernel developers, in whose code STANSE has found bugs, have also expressed interest in using STANSE regularly¹.

B Integration of New Checker

Here we want to show how easy is to integrate a new checker into STANSE. The process requires only two simple step to gain the full functionality provided by STANSE framework.

The first step is to create a subclass, say **MyChecker**, of STANSE abstract class **Checker**. In the subclass we need to implements the following three methods:

- **getName** : This method returns unique name of the checker. It does not have to be readable for humans. But typical implementation is **MyChecker.class.getName()**.
- **toString** : This method returns a short textual description of the checker.
- **check** : This is the method, where the checker's analysis should be implemented in. More precisely, this method is called by STANSE, when checker's analysis have to be performed.

And the second step is to integrate **MyChecker** into STANSE framework. Instances of checkers (already integrated into the STANSE framework) are then created following concrete factory design pattern. This implies for checkers, they have to implement mechanism of creation of their own instances. In STANSE we require to define whole subclass of abstract class **CheckerCreator** instead of definition of a single creation function. But the task is also very simple. In our case we create a subclass **MyCheckerCreator** of **CheckerCreator** abstract class and we implement the following methods:

- **getCheckerName** : This method returns an user-readable name of the checker. So we can implement this method like **return "MyChecker"**
- **getCheckerCreationInfo** : The method returns a comprehensive textual description of the checker. The information is printed to the users of STANSE when they decide, which checkers to use for analysing their code.
- **getDataFilesExtensions** : When our checker requires some configuration files to be loaded, this method returns a list of extensions of those files. When the checker does not require any configuration files, the function returns just empty list. This is also the case for **MyChecker**.

¹ <http://lkm1.org/lkm1/2009/7/18/59> and follow-ups

- `checkArgumentList` : This method accepts a list of files and returns true only iff they represent all configuration files the checker need to work properly. When the checker does not require any configuration files, the method always returns `true`. So implementation in case of `MyChecker` is obvious.
- `createInterprocedural` : This method is supposed to create an instance of the checker. In our case the implementation would be like `return new MyChecker()`.
- `createIntraprocedural` : This method does exactly the same job as the previous one. It is here to allow different checkers to distinguish different implementations for iter- and intra-procedural analyses. Nevertheless, there is typically only a single implementation, so both methods return an instance of the same checker class. This is also the case for `MyChecker`.

We are almost done. The last thing is to register the class `MyCheckerCreator` into STANSE checker creators factory. It consists of appending a line

```
registerCheckerCreator(new MyCheckerCreator())
```

at the end of STANSE's file `CheckerFactory.java`.

C Important Bugs found by Stanse

Although checkers currently implemented in STANSE are based on techniques published many years ago, running then on the Linux kernel helped to uncover several important bugs in the software. To get a better view, how such error look like, we present here two such an errors, discovered by two different checkers.

C.1 AutomatonChecker

Many bugs found by the `AUTOMATONCHECKER` trigger only under specific conditions, however some of them may be visible to the user. Consider this code excerpt taken from the 2.6.27 kernel, `drivers/pci/hotplug/pciehp_core.c` file, `set_lock_status` function:

```
mutex_lock(&slot->ctrl->crit_sect);
/* has it been >1 sec since our last toggle? */
if ((get_seconds() - slot->last_emi_toggle) < 1)
    return -EINVAL;
```

Note the `mutex_lock` function followed by `if` with `return` in the true branch. There is an omitted `mutex_unlock`. STANSE helped to find out this problem. In fact this deadlock could be easily triggered by a user. It is sufficient to write "1" to `/sys/bus/pci/slots/.../lock` file twice within a second.

C.2 ThreadChecker

An example of non-trivial error which could not be found by the `AUTOMATONCHECKER`. The code described here is from the 2.6.28 kernel, file `fs/ecryptfs/messaging.c`.

There are three locks in the code, `msg_ctx->mux` local per context, and two global – `ecryptfs_daemon_hash_mux` and `ecryptfs_msg_ctx_lists_mux`.

Let us denote lock dependencies as a binary relation where the first component depends on the second. I.e. `lock(A)` followed by `lock(B)` means dependency B on A: (A, B) . For short, it is $A \leftarrow B$.

```

1  int  encryptfs_process_response (...)
2  {
3      ...
4      mutex_lock(&msg_ctx->mux);
5      mutex_lock(&encryptfs_daemon_hash_mux);
6      ...
7      mutex_unlock(&encryptfs_daemon_hash_mux);
8      ...
9  unlock:
10     mutex_unlock(&msg_ctx->mux);
11     out:
12     return rc;
13 }

```

Here the two locks on lines 4 and 5 give $\text{msg_ctx->mux} \leftarrow \text{encryptfs_daemon_hash_mux}$.

```

14 static int  encryptfs_send_message_locked (...)
15 {
16     ...
17     mutex_lock(&encryptfs_msg_ctx_lists_mux);
18     ...
19     mutex_unlock(&encryptfs_msg_ctx_lists_mux);
20     ...
21 }
22
23 int  encryptfs_send_message (...)
24 {
25     int rc;
26
27     mutex_lock(&encryptfs_daemon_hash_mux);
28     rc = encryptfs_send_message_locked (...)
29     mutex_unlock(&encryptfs_daemon_hash_mux);
30     return rc;
31 }

```

At line 28, function `encryptfs_send_message_locked` is called from `encryptfs_send_message`, hence the locks at lines 17 and 27 generate lock dependency of `encryptfs_daemon_hash_mux` $\leftarrow$ `encryptfs_msg_ctx_lists_mux`.

```

32 int  encryptfs_wait_for_response (...)
33 {
34     ...
35     mutex_lock(&encryptfs_msg_ctx_lists_mux);
36     mutex_lock(&msg_ctx->mux);
37     ...
38     mutex_unlock(&msg_ctx->mux);
39     mutex_unlock(&encryptfs_msg_ctx_lists_mux);
40     return rc;
41 }

```

Finally, this function introduces $\text{encryptfs_msg_ctx_lists_mux} \leftarrow \text{msg_ctx->mux}$.

Composing these results together the following circular dependency of these three locks was found:

- $\text{msg_ctx->mux} \leftarrow \text{encryptfs_daemon_hash_mux}$
- $\text{encryptfs_daemon_hash_mux} \leftarrow \text{encryptfs_msg_ctx_lists_mux}$
- $\text{encryptfs_msg_ctx_lists_mux} \leftarrow \text{msg_ctx->mux}$

This issue was later confirmed as a real bug leading to a deadlock².

² <http://lkml.org/lkml/2009/4/14/527>